

**Evaluasi Model Transformer from Scratch dan MarianMT
pada Penerjemahan Subtitle Film Bahasa Inggris ke Bahasa
Indonesia**

Group Project: Text Mining



NIM - Name:

- 1. 2702321466 – Glenn Putra Laymando**
- 2. 2702337086 – Theo Filus Iglesias Triadi**
- 3. 2702353865 - Rayyan**

Class Program: Computer Science and Mathematics

Lecturer:

Lili Ayu Wulandhari, S.Si., M.Sc., Ph.D.

**UNIVERSITAS BINA NUSANTARA
COMPUTER SCIENCE
2025, EVEN SEMESTER**

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Perkembangan Natural Language Processing (NLP) telah mendorong kemajuan yang cukup besar dalam bidang *machine translation*. Salah satu pendekatan yang banyak digunakan saat ini adalah Neural Machine Translation (NMT) berbasis arsitektur Transformer yang mampu menangkap hubungan antar kata secara lebih efektif dibandingkan metode-metode yang ada sebelumnya.

Dalam pengembangan sistem penerjemahan, terdapat dua pendekatan utama yang sering digunakan. Pendekatan pertama adalah membangun model Transformer dari awal (Transformer from Scratch) menggunakan dataset yang tersedia. Pendekatan ini memberikan fleksibilitas tinggi, namun membutuhkan data yang cukup besar dan waktu pelatihan yang lebih lama agar mampu menghasilkan performa yang baik. Pendekatan kedua adalah memanfaatkan model *pre-trained* seperti MarianMT yang telah dilatih pada korpus paralel berskala besar dan kemudian disesuaikan kembali melalui proses *fine-tuning*.

Meskipun model *pre-trained* umumnya menghasilkan performa yang lebih baik, perbandingan langsung antara Transformer from Scratch dan MarianMT pada dataset subtitle film Bahasa Inggris-Indonesia masih menarik untuk diteliti. Dataset subtitle dipilih karena mengandung percakapan sehari-hari yang lebih *natural* sehingga dapat merepresentasikan penggunaan bahasa dalam situasi nyata.

Oleh karena itu, proyek ini bertujuan untuk membandingkan performa Transformer from Scratch dan MarianMT Fine-Tuning dalam tugas penerjemahan Bahasa Inggris ke Bahasa Indonesia menggunakan dataset subtitle film. Perbandingan dilakukan berdasarkan proses pelatihan, nilai loss, BLEU Score, dan kualitas hasil terjemahan yang dihasilkan.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah pada proyek ini adalah:

1. Bagaimana proses preprocessing dan representasi teks yang tepat untuk membangun sistem penerjemahan Bahasa Inggris–Indonesia menggunakan dataset subtitle film?
2. Bagaimana performa model Transformer from Scratch dalam melakukan penerjemahan Bahasa Inggris ke Bahasa Indonesia?
3. Bagaimana performa model MarianMT yang di-fine-tune menggunakan dataset subtitle film?
4. Model manakah yang memberikan hasil penerjemahan terbaik berdasarkan evaluasi BLEU Score dan analisis kualitas terjemahan?

1.3 Tujuan Proyek

Tujuan dari proyek ini adalah:

1. Melakukan preprocessing dan persiapan dataset subtitle film Bahasa Inggris-Indonesia.
2. Membangun model penerjemahan menggunakan arsitektur Transformer from Scratch.
3. Melakukan fine-tuning model MarianMT pada dataset yang sama.
4. Mengevaluasi performa kedua model menggunakan BLEU Score dan analisis hasil terjemahan.
5. Membandingkan kelebihan dan kekurangan kedua pendekatan dalam tugas machine translation Bahasa Inggris-Indonesia.

1.4 Jenis Task Machine Learning

Proyek ini termasuk dalam bidang Natural Language Processing (NLP) dengan task Machine Translation atau Sequence-to-Sequence Learning. Input model berupa kalimat dalam Bahasa Inggris, sedangkan output yang diharapkan adalah kalimat terjemahan dalam Bahasa Indonesia. Contohnya:

Tabel 1.1 Contoh Pasangan Kalimat Inggris-Indonesia

Input (English)	Output (Indonesian)
How are you?	Apa kabar?
I will call you later	Aku akan menelponmu nanti

Task ini berbeda dengan klasifikasi maupun regresi karena model menghasilkan urutan kata (sequence generation), bukan label ataupun nilai numerik.

1.5 Dataset

Dataset yang digunakan berasal dari Kaggle: [English - Indonesia Movie Subtitles](#). Dataset berisi pasangan kalimat Bahasa Inggris dan Bahasa Indonesia yang berasal dari subtitle film. Untuk mengurangi beban komputasi, dilakukan random sampling sebanyak 100.000 pasangan kalimat dari dataset asli. Dataset memiliki dua atribut utama:

Tabel 1.2 Struktur Dataset

Kolom	Deskripsi
en	Kalimat Bahasa Inggris
id	Kalimat Bahasa Indonesia

Hasil pemeriksaan awal menunjukkan:

- Jumlah data: 100.000 pasangan kalimat.
- Missing value: 0
- Duplicate row (en-id): 13 data
- Empty string: 0
- Tipe data seluruh kolom berupa string.

1.6 Analisis Dataset (EDA)

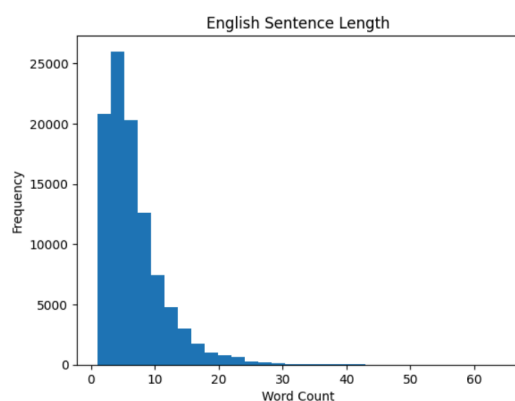
1.6.1 Distribusi Panjang Kalimat

Tabel 1.3 Statistik Dataset

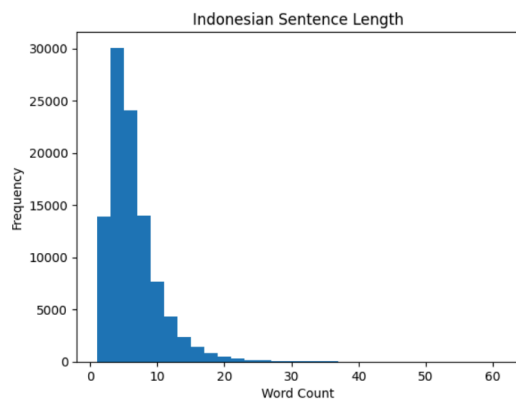
Statistik	English	Indonesia
Mean	6.88 kata	5.87 kata
Median	6 kata	5 kata
Maksimum	64 kata	61 kata

Berdasarkan hasil analisis distribusi panjang kalimat, sebagian besar pasangan kalimat memiliki panjang kurang dari 25 kata baik pada sisi Bahasa Inggris maupun Bahasa Indonesia. Jumlah yang lebih dari 25 hanya 678 baris.

Gambar 1.1 Histogram Panjang Kalimat Bahasa Inggris

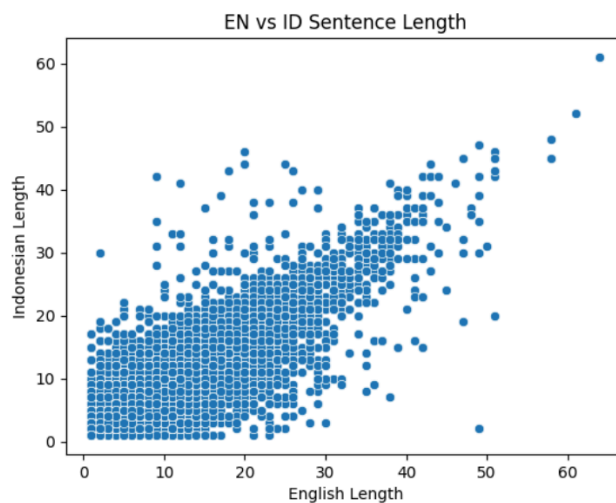


Gambar 1.2 Histogram Panjang Kalimat Bahasa Indonesia



1.6.2 Hubungan Panjang Kalimat Inggris dan Indonesia

Gambar 1.3 Scatter Plot Panjang Kalimat Indonesia dan Inggris



Berdasarkan hasil perhitungan korelasi panjang kalimat diperoleh: Correlation = 0.876. Nilai tersebut menunjukkan korelasi positif yang kuat antara panjang kalimat Bahasa Inggris dan Bahasa Indonesia. Semakin panjang kalimat sumber, semakin panjang juga kalimat hasil terjemahannya. Berdasarkan scatter plot, terlihat pola linear yang cukup jelas sehingga pasangan kalimat dalam dataset relatif konsisten.

1.6.3 Kosakata Dataset

Jumlah kosakata unik yang ditemukan:

Tabel 1.4 Jumlah Kosakata Unik

Bahasa	Vocabulary Size
English	68.086
Indonesian	73.410

Kosakata Bahasa Indonesia sedikit lebih banyak dibandingkan Bahasa Inggris. Hal ini menunjukkan variasi kata yang cukup tinggi dari hasil terjemahan subtitle.

Kata-kata yang sering muncul pada data Bahasa Inggris:

- the
- I
- to
- you
- a

Sedangkan pada Bahasa Indonesia

- yang
- aku
- kau
- di
- akan

Frekuensi kata tersebut sesuai dengan karakteristik dialog percakapan sehari-hari.

1.6.4 Kualitas Data

Hasil analisis menemukan beberapa karakteristik berikut:

- Tidak terdapat missing value.
- Tidak terdapat empty string.
- Terdapat beberapa pasangan kalimat dengan perbedaan panjang yang cukup ekstrim
- Terdapat karakter khusus, simbol subtitle, dan noise hasil ekstraksi subtitle.
- Terdapat sejumlah kecil pasangan kalimat yang terdeteksi tidak sesuai bahasa berdasarkan language detection.

-

1.6.5 Verifikasi Bahasa

Untuk memeriksa konsistensi bahasa dilakukan language detection pada setiap kalimat menggunakan library pendeteksi bahasa. Hasil pemeriksaan menunjukkan masih terdapat 10.627 pasangan kalimat yang terdeteksi mencurigakan. Namun setelah dilakukan pengecekan manual ditemukan bahwa sebagian besar kasus terjadi karena: kalimat sangat pendek, nama orang, singkatan, kata yang mirip antarbahasa, dan keterbatasan algoritma language detection.

Contoh: what do we do?

terdeteksi sebagai ph (Polish), meskipun sebenarnya merupakan kalimat bahasa inggris yang valid. Oleh karena itu, data tetap dipertahankan.

1.6.6 Analisis Noise dan Anotasi Subtitle

Selain melakukan analisis panjang kalimat dan distribusi kosakata, dilakukan juga pemeriksaan terhadap keberadaan anotasi subtitle yang ditandai dengan tanda kurung siku ([]). Contoh data yang ditemukan:

Tabel 1.5 Anotasi Subtitle

English	Indonesian
[BEEPING]	[BIP]
[electronic chirping]	[Kicau elektronik]
[Granny] If you want privacy, talk fast.	cepatlah bicara.
I think the shapeshifter is its dad. [Wails]	Saya pikir shapeshifter adalah ayahnya.

Hasil identifikasi menunjukkan terdapat 274(inggris) dan 237(indonesia) pasangan kalimat yang mengandung anotasi dalam tanda kurung siku. Anotasi tersebut umumnya merepresentasikan:

- Efek suara (sounds effect)
- ekspresi non-verbal
- identitas pembicara
- petunjuk adegan

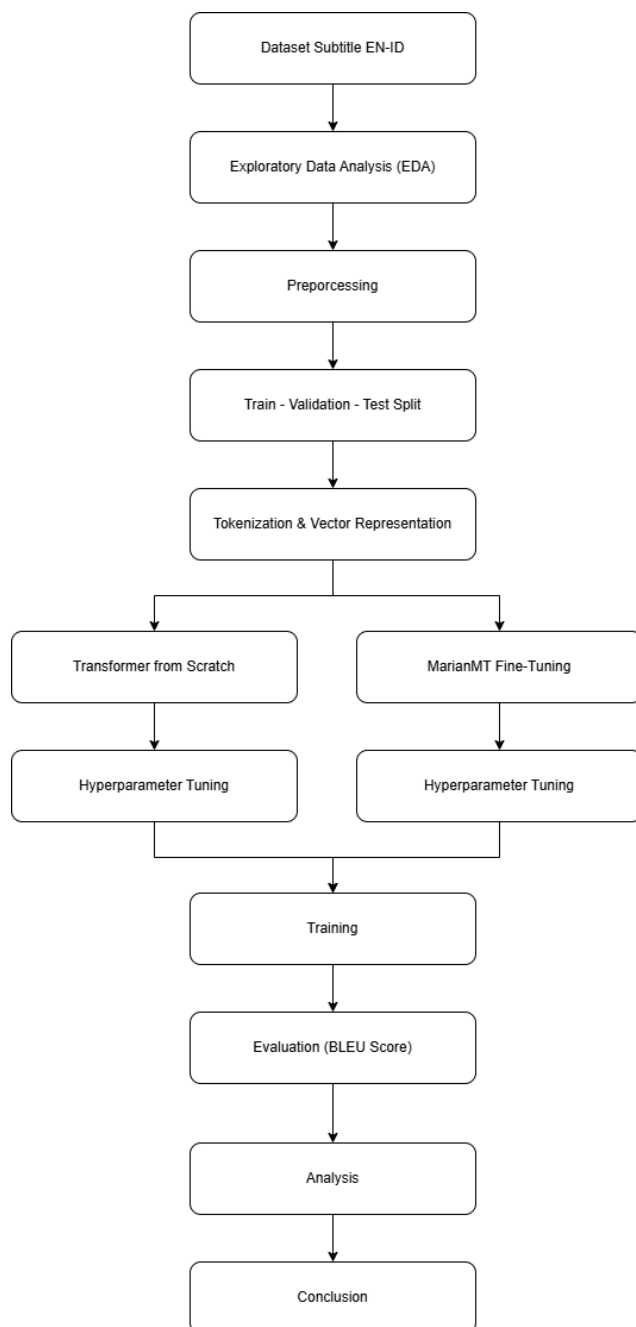
Keberadaan anotasi ini tidak selalu berkontribusi terhadap proses penerjemahan kalimat utama. Bahkan dalam beberapa kasus, anotasi dapat menambah noise karena bukan bagian dari percakapan yang sebenarnya. Meskipun jumlahnya kecil (0.274%) dari keseluruhan dataset, keberadaan anotasi ini perlu diperhatikan pada tahap preprocessing.

BAB 2. METODOLOGI

2.1 Metode Penyelesaian Masalah

Proyek ini dilakukan melalui beberapa tahapan, yaitu eksplorasi data, preprocessing, pembagian dataset, representasi teks, pembangunan model, hyperparameter tuning, pelatihan model, evaluasi menggunakan BLEU Score, analisis hasil penerjemahan, serta kesimpulan. Proses lengkapnya bisa dilihat pada gambar 2.1

Gambar 2.1 Alur Tahapan Penyelesaian Masalah

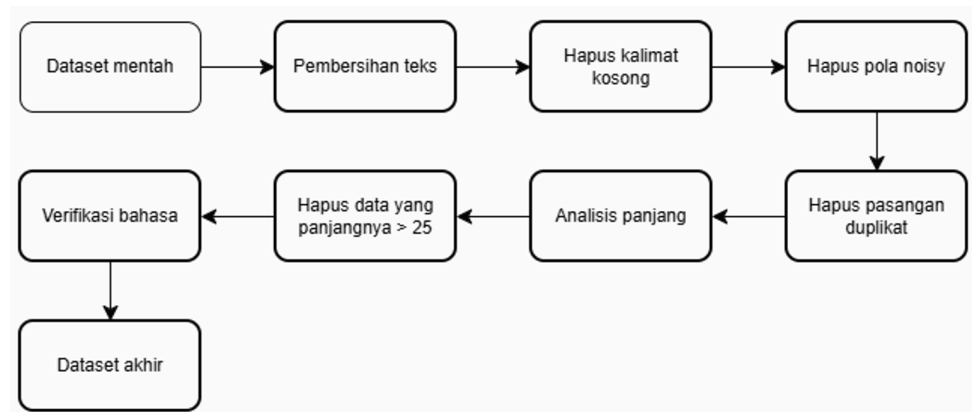


2.2 Preprocessing

2.2.1 Alur Preprocessing

Tahapan preprocessing dilakukan untuk meningkatkan kualitas dataset sebelum digunakan dalam proses pelatihan model translasi. Dataset subtitle film mengandung berbagai bentuk noise seperti anotasi subtitle, karakter khusus, duplikasi data, serta kalimat dengan panjang yang tidak proporsional. Oleh karena itu dilakukan beberapa tahapan pembersihan data seperti pada gambar

Gambar 2.2 Alur Preprocessing



2.2.2 Text Cleaning

Pada tahap ini teks dibersihkan dari berbagai karakter yang tidak relevan terhadap proses translasi. Proses yang dilakukan meliputi

a. Unicode Normalization

Unicode normalization digunakan untuk menyeragamkan bentuk karakter sehingga karakter yang punya arti yang sama memiliki bentuk yang konsisten.

Contoh: Hello menjadi Hello

b. Penghapusan Invisible Characters

Dataset subtitle mengandung beberapa karakter tersembunyi seperti:

- BOM (\uffff)
- Zero Width Space (\u200b)
- Left-to-Right Mark (\u200e)
- Unicode Direction Marker (\u202d)
- Non-breaking Space (\xa0)

Karakter-karakter tersebut dapat mengganggu tokenisasi sehingga dihapus dari dataset.

c. Penghapusan Subtitle Annotations

Hasil EDA menunjukkan terdapat 274 baris data yang mengandung anotasi subtitle dalam tanda kurung siku. Contoh: [BEEPING], [electronic chirping], [Wails] , [Granny]. Anotasi ini umumnya merepresentasikan efek suara, identitas pembicara, petunjuk adegan. Karena tidak termasuk

bagian utama percakapan, anotasi tersebut dihapus menggunakan pola regular expression. Contoh: [Granny] If you want privacy, talk fast.
menjadi: If you want privacy, talk fast.

d. Penghapusan Karakter Kontrol dan Simbol Khusus

Karakter kontrol ASCII, simbol musik subtitle, bullet point, trademark, dan tag HTML dihapus dari teks. Contoh simbol yang dihapus: ♪ ♫ ♬ ♥ ☺TM. Selain itu tag HTML seperti , <div>, juga dihapus.

e. Hapus metadata subtitle

Pola yang dihapus yaitu font color. Contoh: Font color="#ffffff". Karena tidak bermanfaat untuk proses translasi, baris yang mengandung pola tersebut dihapus.

f. Standardisasi Tanda Baca

Berbagai variasi tanda kutip dan dash diseragamkan menjadi format standar.

Contoh:

—

-

—

menjadi

-

g. Case Folding

Seluruh teks diubah menjadi huruf kecil (lowercase)

Contoh: I Love You

menjadi: i love you

Tahapan ini bertujuan mengurangi ukuran kosakata (vocabulary size) sehingga model dapat belajar lebih efisien.

h. Menghapus tanda hubung (-)

Menghapus strip di awal dialog subtitle. Contoh: - Hello menjad Hello.

Selain itu strip yang berdiri sendiri di tengah kalimat juga dihapus.

Contoh: - Jeremy? - Hold on menjad Jeremy? Hold on.

i. Whitespace Normalization

Spasi berlebih yang muncul akibat proses pembersihan teks dirapikan menjadi satu spasi.

Contoh: hello world

menjadi: hello world

2.2.3 Penghapusan Kalimat Kosong

Setelah proses text cleaning, dilakukan pemeriksaan terhadap pasangan kalimat yang menjadi kosong. Hanya pasangan kalimat yang memiliki isi pada kedua sisi (Bahasa Inggris dan Bahasa Indonesia) yang dipertahankan. Tujuan tahap ini adalah menghindari error pada proses tokenisasi dan pelatihan model

2.2.4 Penghapusan Data Duplikat

Duplikasi pasangan kalimat dapat menyebabkan model memberikan bobot berlebih terhadap contoh tertentu. Hasil pemeriksaan menunjukkan terdapat 92 pasangan duplikat setelah cleaning. Oleh karena itu dilakukan penghapusan duplicate pair berdasarkan kombinasi: (English Sentence, Indonesian Sentence).

2.2.5 Analisis Panjang Kalimat

Setelah proses cleaning dilakukan perhitungan ulang statistik panjang kalimat. Dilakukan perhitungan ulang karena setelah proses cleaning isi kalimat sudah ada yang berubah karena anotasi subtitle dihapus, spasi dirapikan, dll. Jadi hasil perhitungan dari EDA sudah tidak akurat lagi. Hasil statistik menunjukkan:

Tabel 2.1 Tabel Statistik Bahasa Indonesia dan Inggris

Statistik	English	Indonesian
Mean	6.77	5.78
Median	6	5
Maximum	64	61

Mayoritas pasangan kalimat memiliki panjang relatif pendek yang sesuai dengan karakteristik subtitle film.

2.2.6 Length Filtering

Berdasarkan distribusi panjang kalimat pada hasil EDA, sebagian besar data berada pada rentang kurang dari 25 kata. Kalimat dengan panjang lebih dari 25 kata dianggap sebagai outlier karena:

- Jumlah sangat sedikit (678 untuk inggris dan 312 untuk indonesia).
- Tidak mewakili distribusi utama dataset.
- Meningkatkan kebutuhan memory model.
- Memperpanjang waktu training.
- Berpotensi mengandung kesalahan segmentasi subtitle.

Data yang terhapus berjumlah 721 (Data setelah cleaning: 99773 - data setelah penghapusan > 25: 99052)

2.2.7 Hasil Akhir Preprocessing

Setelah seluruh tahapan preprocessing selesai, diperoleh dataset akhir sebanyak 99.129 pasangan kalimat.

Tabel 2.2 Tabel Hasil Preprocessing

Parameter	Nilai
Total pasangan kalimat	99.052
Rata-rata panjang English	6,60 kata
Rata-rata panjang Indonesian	5,65 kata
Maksimum panjang English	25 kata
Maksimum panjang Indonesian	25 kata
Rata-rata rasio panjang	1,25
Rata-rata selisih panjang	1,50

2.2.8 Train Test Splitting

Sebelum proses pelatihan model dilakukan, dataset dibagi menjadi tiga bagian, yaitu data pelatihan (*training set*), data validasi (*validation set*), dan data pengujian (*test set*). Pembagian ini bertujuan untuk memastikan bahwa model dapat dilatih, dievaluasi, dan diuji menggunakan data yang berbeda sehingga performa yang diperoleh dapat merepresentasikan kemampuan generalisasi model terhadap data yang belum pernah dilihat sebelumnya.

Proses pembagian dilakukan dalam dua tahap. Pada tahap pertama, dataset dibagi menjadi data pelatihan sebesar 80% dan data sementara sebesar 20%. Selanjutnya, data sementara tersebut dibagi kembali secara merata menjadi data validasi dan data pengujian, masing-masing sebesar 10% dari keseluruhan dataset. Dengan demikian, proporsi akhir pembagian data adalah 80% untuk pelatihan, 10% untuk validasi, dan 10% untuk pengujian.

Data pelatihan digunakan untuk memperbarui parameter model selama proses training. Data validasi digunakan untuk mengevaluasi performa model selama pelatihan, termasuk pada proses hyperparameter tuning dan mekanisme early stopping. Sementara itu, data pengujian digunakan pada tahap akhir proyek untuk mengukur performa model secara objektif terhadap data yang tidak pernah digunakan selama proses pelatihan maupun validasi.

Tabel 2.3 Pembagian Proporsi Dataset

Dataset	Jumlah Data
Training Set	79241
Validation Set	9905
Test Set	9906

2.3 Vector Representation

Vector representation merupakan proses mengubah data teks menjadi representasi numerik yang dapat diproses oleh model machine learning maupun deep learning. Transformer tidak dapat memahami kata atau kalimat secara langsung, sehingga setiap token harus direpresentasikan dalam bentuk vektor berdimensi tertentu yang mengandung informasi yang dapat dipahami transformer.

Pada model berbasis Transformer, proses vector representation umumnya dilakukan melalui tahap tokenisasi dan embedding. Tokenisasi bertujuan untuk memecah kalimat menjadi unit-unit yang lebih kecil berupa token, kemudian setiap token dikonversi menjadi token ID berdasarkan vocabulary yang digunakan. Token ID tersebut selanjutnya dipetakan menjadi vector yang menggunakan embedding layer sehingga token yang memiliki makna serupa cenderung memiliki representasi vektor yang berdekatan.

Selain token embedding, arsitektur Transformer juga menggunakan positional embedding untuk mempertahankan informasi urutan kata dalam kalimat. Hal ini diperlukan karena mekanisme self-attention pada Transformer tidak secara inheren memperhatikan posisi token. Representasi akhir suatu token diperoleh dengan menggabungkan token embedding dan positional embedding sebelum diteruskan ke lapisan encoder maupun decoder.

Dalam proyek ini, proses vector representation dilakukan menggunakan dua pendekatan yang berbeda. Pada model Transformer from Scratch, embedding dipelajari secara langsung selama proses pelatihan melalui embedding layer yang dibangun dari awal. Sementara itu, pada model MarianMT, representasi vektor

diperoleh dari embedding layer yang telah dipelajari selama proses *pre-training* pada korpus paralel berskala besar. Kedua pendekatan tersebut sama-sama menghasilkan representasi numerik yang digunakan sebagai masukan bagi arsitektur Transformer untuk melakukan masalah translasi mesin.

2.3.1 Tokenization

Tokenization merupakan proses mengubah teks menjadi sekumpulan token yang dapat diproses oleh model pembelajaran mesin. Pada proyek ini, proses tokenisasi dilakukan menggunakan MarianTokenizer yang merupakan tokenizer bawaan dari model MarianMT. Tokenizer ini menggunakan algoritma SentencePiece yang menerapkan pendekatan subword tokenization. Sebagai contoh, kalimat:

"I love machine translation"

akan dipecah menjadi beberapa token sebelum dikonversi menjadi indeks numerik (token IDs) yang sesuai dengan kosakata (vocabulary) model. Token ID tersebut kemudian digunakan sebagai masukan pada tahap vector representation. Pada kasus sebelumnya, akan diubah menjadi :

[_I, _love, _machine, _translation]

Pendekatan subword tokenization memungkinkan suatu kata dipecah menjadi bagian-bagian yang lebih kecil sehingga dapat mengurangi permasalahan out-of-vocabulary (OOV). Misalkan, terdapat kata:

"internationalization"

dapat direpresentasikan menjadi beberapa subword seperti:

[_international, ization]

Dengan pendekatan ini, kata-kata yang jarang muncul atau tidak terdapat dalam kosakata tetap dapat direpresentasikan melalui kombinasi beberapa subword yang telah dikenal oleh model.

Pada proyek ini, MarianTokenizer digunakan baik pada model Transformer yang dibangun dari awal maupun pada model MarianMT yang di-fine-tune. Dengan demikian, kedua model menggunakan proses tokenisasi yang sama sehingga perbedaan performa yang diperoleh lebih merepresentasikan perbedaan arsitektur dan proses pembelajaran model dibandingkan perbedaan pada tahap tokenisasi.

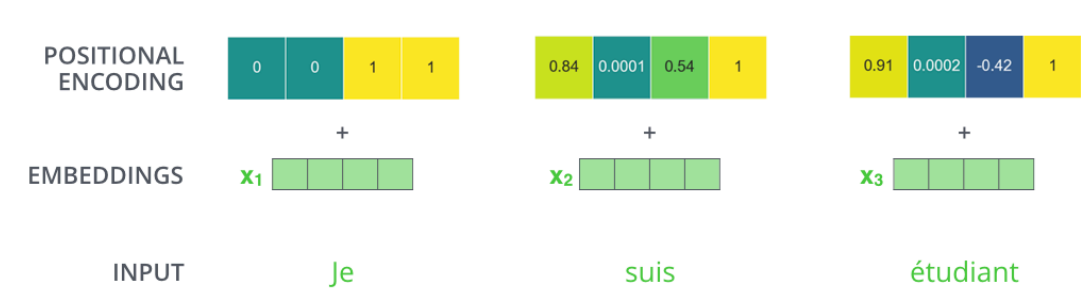
2.3.2 Embedding Representation

Setelah proses tokenisasi, setiap token direpresentasikan dalam bentuk indeks numerik (*token IDs*). Namun, representasi tersebut belum dapat secara langsung digunakan oleh model transformer karena belum memiliki mengandung informasi

mengenai hubungan antar kata. Oleh karena itu, diperlukan proses *embedding* untuk memetakan setiap token ke dalam vektor berdimensi tetap.

Embedding merupakan teknik representasi kata yang mengubah token menjadi vektor numerik sehingga kata-kata yang memiliki makna yang mirip cenderung memiliki representasi yang berdekatan dalam vektor. Melalui proses *training*, model dapat menangkap hubungan makna, sintaks, dan konteks antar token berdasarkan pola yang terdapat pada data *training*.

Gambar 2.2 Transformer Embedding



Gambar 2.2 menunjukkan proses pembentukan representasi vektor pada arsitektur Transformer. Setiap token hasil tokenisasi terlebih dahulu dipetakan menjadi token embedding yang merepresentasikan makna token dalam ruang vektor. Karena Transformer tidak memiliki mekanisme bawaan untuk memahami urutan token, ditambahkan positional embedding yang berfungsi memberikan informasi posisi setiap token dalam kalimat.

Dalam proyek ini, proses embedding dilakukan dengan dua pendekatan yang berbeda sesuai dengan arsitektur model yang digunakan.

2.3.2.1 Transformer from Scratch

Pada model Transformer from Scratch, token ID hasil tokenisasi dipetakan ke representasi vektor menggunakan layer embedding dengan dimensi 256. Embedding layer ini dipelajari selama proses *training* dan menghasilkan representasi makna untuk setiap token dalam kosakata. Selain itu, model juga menggunakan positional embedding untuk merepresentasikan posisi token dalam urutan kalimat. Representasi yang diberikan kepada encoder diperoleh melalui penjumlahan token embedding dan positional embedding sehingga model dapat memahami baik makna token maupun posisinya dalam kalimat.

Dimensi embedding ditetapkan sebesar 256. Nilai ini dipilih sebagai jalan tengah antara kapasitas representasi dan efisiensi komputasi. Dimensi yang lebih kecil berpotensi membatasi kemampuan model dalam merepresentasikan hubungan

makna antar token, sedangkan dimensi yang terlalu besar meningkatkan kebutuhan memori dan waktu training. Dengan ukuran dataset sekitar 100.000 pasangan kalimat serta keterbatasan sumber daya GPU dan RAM yang tersedia, dimensi 256 lebih cocok karena mampu memberikan representasi yang cukup lumayan dengan biaya komputasi yang masih bisa ditangani.

Dengan pendekatan ini, representasi kata sepenuhnya dipelajari dari data training yang digunakan pada proyek sehingga kualitas embedding sangat bergantung pada data yang tersedia.

2.3.2.2 MarianMT Fine-Tuning

Pada model MarianMT, proses embedding dilakukan menggunakan *pre-trained embedding layer* yang telah dipelajari sebelumnya selama proses *pre-training* pada korpus paralel berskala besar. Setelah proses tokenisasi menghasilkan *input_ids*, setiap token secara otomatis dipetakan ke representasi vektor menggunakan parameter embedding yang telah tersedia pada model *pre-trained* sebelum diproses lebih lanjut oleh encoder dan decoder.

Penggunaan *pre-trained embedding* memungkinkan model memanfaatkan pengetahuan makna ataupun struktur yang telah dipelajari dari data dalam jumlah besar sehingga dapat menghasilkan representasi makna yang lebih kaya dibandingkan embedding yang dilatih dari awal. Berbeda dengan model Transformer from Scratch yang membangun representasi vektor melalui embedding layer yang dipelajari sepenuhnya dari dataset, MarianMT mewarisi representasi vektor yang telah terbentuk selama proses *pre-training*.

2.4 Hyperparameter Tuning

Hyperparameter tuning merupakan proses pencarian kombinasi parameter terbaik yang digunakan sebelum proses pelatihan model. Berbeda dengan parameter model yang dipelajari secara otomatis selama proses training, hyperparameter ditentukan oleh peneliti dan dapat memengaruhi performa, stabilitas, serta kemampuan generalisasi model. Pada proyek ini, hyperparameter tuning dilakukan menggunakan metode Grid Search, yaitu pendekatan yang mengevaluasi seluruh kombinasi parameter yang telah ditentukan sebelumnya. Kombinasi terbaik dipilih berdasarkan nilai validation loss terendah pada data validasi.

2.4.1 Hyperparameter Tuning Transformer from Scratch

Pada model Transformer yang dibangun dari awal, proses hyperparameter tuning dilakukan untuk menentukan jumlah layer Transformer (*num_layers*) dan learning

rate yang menghasilkan performa terbaik. Pencarian dilakukan menggunakan Grid Search dengan kombinasi parameter yang ditunjukkan pada Tabel 2.4.

Tabel 2.4 Kombinasi Hyperparameter Tuning Transformer from Scratch

Hyperparameter	Nilai yang Diuji
Number of Layers	2, 4, 8
Learning Rate	0.0001, 0.0005, 0.001

Setiap kombinasi parameter dilatih selama maksimal 5 epoch menggunakan optimizer Adam dan fungsi loss sparse categorical crossentropy. Untuk mencegah overfitting dan mempercepat proses pelatihan, digunakan mekanisme Early Stopping dengan nilai patience sebesar 2. Evaluasi dilakukan berdasarkan nilai validation loss terendah yang diperoleh selama proses pelatihan.

2.4.2 Hyperparameter Tuning MarianMT Fine-Tuning

Pada model MarianMT, proses hyperparameter tuning difokuskan pada learning rate dan weight decay yang digunakan selama proses *fine-tuning*. Parameter tersebut dipilih karena memiliki pengaruh besar terhadap kemampuan model dalam beradaptasi terhadap dataset baru tanpa menghilangkan pengetahuan yang telah diperoleh selama *pre-training*. Grid Search dilakukan dengan kombinasi parameter yang ditunjukkan pada Tabel 2.5.

Tabel 2.5 Kombinasi Hyperparameter Tuning Transformer MarianMT Fine-Tuning

Hyperparameter	Nilai yang Diuji
Learning Rate	0.00001, 0.00003, 0.00005
Weight Decay	0.00, 0.01, 0.10

Setiap kombinasi dievaluasi dengan menggunakan HuggingFace Seq2SeqTrainer dengan jumlah epoch sebanyak 2. Pemilihan model terbaik dilakukan berdasarkan nilai evaluation loss terendah pada data validasi.

2.5 Modeling

Pada proyek ini digunakan dua pendekatan penerjemahan bahasa berbasis Transformer, yaitu Transformer yang dibangun dari awal (*Transformer from Scratch*) dan model *pretrained* MarianMT yang di-*fine-tune* menggunakan dataset proyek. Kedua model menerima masukan berupa token hasil proses tokenisasi dan embedding, kemudian menghasilkan kalimat terjemahan dalam bahasa target melalui mekanisme encoder-decoder.

2.5.1 Transformer from Scratch

Model pertama merupakan implementasi arsitektur Transformer yang dibangun dari awal menggunakan TensorFlow dan Keras. Arsitektur ini terdiri atas encoder dan decoder yang saling terhubung melalui mekanisme attention.

Gambar 2.3 Arsitektur Transformer from Scratch



Pada gambar 2.3, arsitektur Transformer from Scratch yang digunakan terdiri atas dua encoder layer dan dua decoder layer (Hasil dari *grid search*). Setiap encoder layer menerapkan Multi-Head Self-Attention dan Feed Forward Network (FFN), sedangkan setiap decoder layer terdiri atas Masked Multi-Head Self-Attention, Cross-Attention, dan Feed Forward Network. Seluruh layer menggunakan residual connection dan layer normalization untuk meningkatkan stabilitas proses pelatihan.

Pada bagian encoder, token hasil embedding diproses menggunakan beberapa lapisan encoder yang terdiri dari Multi-Head Self-Attention dan Feed Forward Network (FFN). Mekanisme self-attention memungkinkan model mempelajari hubungan antar token dalam satu kalimat secara paralel sehingga mampu menangkap konteks global dari kalimat sumber.

Setiap encoder layer menerapkan residual connection dan layer normalization untuk meningkatkan stabilitas proses *training* serta mengurangi masalah *vanishing gradient*. Pada bagian decoder, model menggunakan Masked Multi-Head Self-Attention untuk memastikan bahwa token yang sedang diprediksi hanya dapat mengakses token-token sebelumnya. Selain itu, decoder juga menerapkan Cross-Attention yang memungkinkan informasi dari encoder digunakan selama proses prediksi terjemahan. Feed Forward Network digunakan setelah proses attention untuk melakukan transformasi non linear terhadap

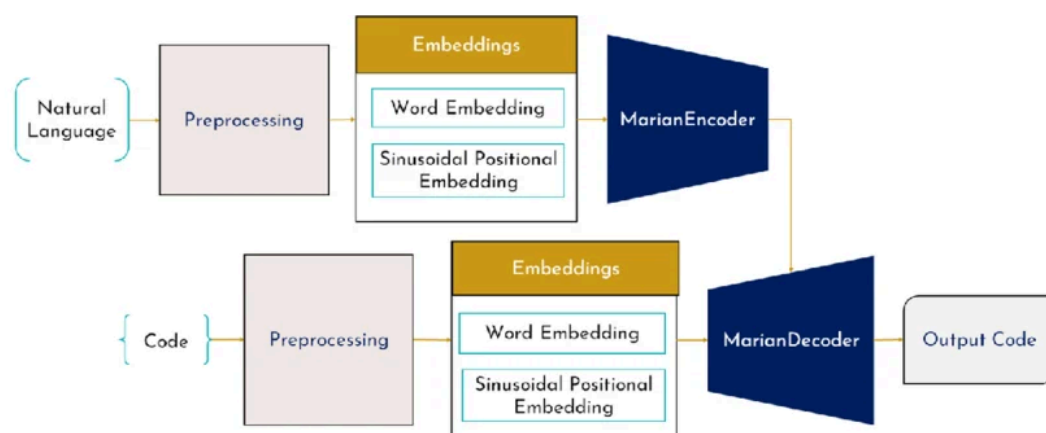
representasi yang dihasilkan. Lapisan ini terdiri atas dua fully connected layer dengan fungsi aktivasi ReLU.

Berdasarkan hasil hyperparameter tuning, konfigurasi terbaik diperoleh dengan jumlah encoder-decoder layer sebanyak 2 dan learning rate 0,0005. Selain itu, model menggunakan konfigurasi tambahan dimensi embedding 256, jumlah attention head sebanyak 8, dan dimensi feed forward sebesar 512. Model dilatih menggunakan optimizer Adam dengan fungsi loss sparse categorical crossentropy. Untuk mengurangi risiko overfitting digunakan Early Stopping dengan nilai patience sebesar 5.

2.5.2 MarianMT Fine-Tuning

Model kedua menggunakan MarianMT, yaitu model Transformer encoder-decoder *pre-trained* yang telah dilatih pada korpus paralel berskala besar untuk masalah penerjemahan bahasa. Berbeda dengan Transformer from Scratch yang membangun seluruh parameter dari awal, MarianMT menggunakan bobot model yang telah dipelajari selama proses *pre-training*.

Gambar 2.4 Arsitektur MarianMT



Gambar 2.4 menunjukkan alur pemrosesan pada model MarianMT yang digunakan dalam proyek ini. Kalimat sumber terlebih dahulu melalui tahap tokenisasi menggunakan MarianTokenizer berbasis SentencePiece. Hasil tokenisasi kemudian dikonversi menjadi representasi vektor melalui embedding layer *pre-trained* yang telah tersedia pada model MarianMT. Selanjutnya, representasi tersebut diperkaya dengan informasi posisi token sebelum diproses oleh encoder.

Encoder sendiri bertugas untuk membangun representasi konteks dari kalimat sumber menggunakan mekanisme multi-head self-attention. Representasi yang dihasilkan encoder kemudian diteruskan ke decoder melalui mekanisme

cross-attention untuk menghasilkan token terjemahan secara bertahap sampai terbentuk kalimat target.

Berbeda dengan Transformer from Scratch yang mempelajari seluruh parameter dari awal menggunakan dataset proyek, MarianMT memanfaatkan parameter encoder, decoder, dan embedding yang telah dipelajari sebelumnya selama proses pre-training. Selanjutnya parameter tersebut disesuaikan kembali melalui proses fine-tuning agar lebih sesuai dengan karakteristik dataset penerjemahan yang digunakan.

Pada tahap *fine-tuning*, parameter model disesuaikan kembali menggunakan dataset proyek sehingga model dapat beradaptasi terhadap karakteristik data yang digunakan. Proses pelatihan dilakukan menggunakan framework HuggingFace Transformers melalui kelas Seq2SeqTrainer yang menyediakan mekanisme pelatihan, evaluasi, dan penyimpanan model secara otomatis.

Berdasarkan hasil hyperparameter tuning, konfigurasi terbaik diperoleh dengan learning rate sebesar 0,00001 dan weight decay sebesar 0,00. Model dilatih selama maksimal 20 epoch dengan batch size 8 untuk data pelatihan dan validasi. Selain itu, digunakan Early Stopping dengan nilai patience sebesar 3 untuk menghentikan pelatihan apabila tidak terjadi perbaikan performa pada data validasi. Selama proses pelatihan, model terbaik dipilih berdasarkan nilai evaluation loss terendah menggunakan mekanisme `load_best_model_at_end`.

BAB 3. Hasil dan Analisa

3.1 Hasil Hyperparameter Tuning

Pada tahap ini dilakukan evaluasi terhadap berbagai kombinasi hyperparameter yang telah ditentukan pada Bab Metodologi menggunakan metode Grid Search. Tujuan dari proses ini adalah untuk memperoleh konfigurasi parameter terbaik yang mampu menghasilkan performa terbaik berdasarkan nilai *validation loss* untuk Transformer from Scratch dan *evaluation loss* untuk MarianMT. Hasil tuning ini akan digunakan untuk menentukan konfigurasi model yang akan digunakan pada tahap pelatihan dan evaluasi selanjutnya.

3.1.1 Hasil Hyperparameter Tuning Transformer from Scratch

Untuk model Transformer from Scratch, proses Grid Search dilakukan dengan menguji kombinasi jumlah layer (*num_layers*) dan *learning rate* yang berbeda. Setiap kombinasi dievaluasi menggunakan nilai *validation loss* pada data validasi. Hasil lengkap proses tuning ditunjukkan pada Gambar 3.1.

Gambar 3.1 Hasil Grid Search Tuning Transformer from Scratch

	num_layers	learning_rate	best_val_loss
1	2	0.0005	1.147640
2	2	0.0010	1.228854
4	4	0.0005	1.301115
0	2	0.0001	1.382630
3	4	0.0001	1.703143
5	4	0.0010	1.714796
6	8	0.0001	1.722801
7	8	0.0005	1.725750
8	8	0.0010	9.203792

Pada gambar 3.1, hasil Grid Search menunjukkan bahwa kombinasi hyperparameter terbaik diperoleh pada jumlah layer sebanyak 2 dan learning rate sebesar 0,0005. Hasil ini menunjukkan bahwa peningkatan jumlah layer tidak selalu menghasilkan performa yang lebih baik. Hal ini kemungkinan karena model yang lebih dalam cenderung tidak memberikan peningkatan kemampuan generalisasi yang signifikan pada dataset ukuran yang tidak terlalu banyak, sementara learning rate sebesar 0,0005 memberikan keseimbangan yang baik antara kecepatan konvergensi dan stabilitas *training*.

3.1.2 Hasil Hyperparameter Tuning MarianMT Fine-Tuning

Pada model MarianMT, proses Grid Search difokuskan pada pencarian kombinasi *learning rate* dan *weight decay* terbaik selama proses *fine-tuning*. Evaluasi dilakukan menggunakan nilai *evaluation loss* pada data validasi. Hasil tuning ditampilkan pada Gambar 3.2.

Gambar 3.2 Hasil Grid Search Tuning MarianMT

	lr	weight_decay	eval_loss
0	0.00001	0.00	0.448075
1	0.00001	0.01	0.448144
2	0.00001	0.10	0.448685
3	0.00003	0.00	0.482615
4	0.00003	0.01	0.482879
5	0.00003	0.10	0.485109
6	0.00005	0.00	0.522638
7	0.00005	0.01	0.523046
8	0.00005	0.10	0.526809

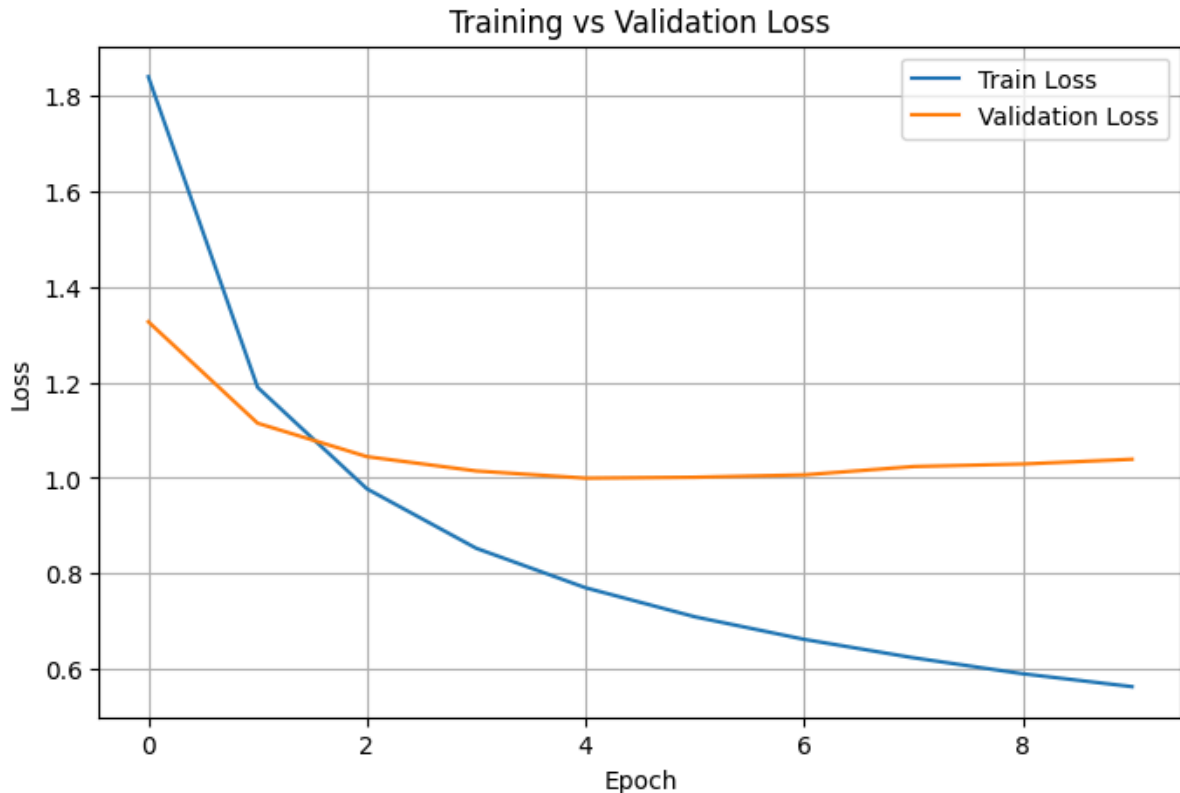
Pada gambar 3.2, hasil Grid Search menunjukkan bahwa kombinasi hyperparameter terbaik diperoleh pada learning rate sebesar 0,00001 dan weight decay sebesar 0,00. Learning rate yang relatif kecil memungkinkan proses *fine-tuning* menyesuaikan parameter model secara lebih stabil tanpa merusak representasi makna yang telah dipelajari selama tahap *pre-training*. Sementara itu, penggunaan weight decay tidak memberikan peningkatan performa yang signifikan pada dataset proyek sehingga nilai terbaik diperoleh ketika regularisasi weight decay tidak digunakan.

3.2 Hasil Pelatihan Model

Pada tahap ini dilakukan analisis terhadap proses pelatihan kedua model menggunakan konfigurasi hyperparameter terbaik yang diperoleh dari tahap Grid Search. Evaluasi dilakukan dengan mengamati perkembangan nilai loss pada data training maupun validation selama proses pelatihan.

3.2.1 Hasil Pelatihan Transformer from Scratch

Gambar 3.3 Grafik Loss Hasil Pelatihan Transformer from Scratch



Berdasarkan hasil pelatihan, model Transformer from Scratch menunjukkan peningkatan performa yang bagus selama beberapa epoch pertama. Nilai training accuracy meningkat dari 67,14% pada epoch pertama menjadi 86,21% pada epoch ke-10. Pada saat yang sama, nilai training loss menurun dari 1,8388 menjadi 0,5635 yang menunjukkan bahwa model berhasil mempelajari pola pada data *training*.

Sementara itu, nilai validation loss mengalami penurunan dari 1,3265 pada epoch pertama menjadi 0,9995 pada epoch ke-5. Setelah epoch ke-5, validation loss mulai meningkat meskipun training loss terus menurun. Kondisi ini mengindikasikan mulai terjadinya overfitting, yaitu model semakin baik dalam mempelajari data *training* tetapi kemampuan generalisasinya terhadap data yang belum pernah dilihat tidak mengalami peningkatan.

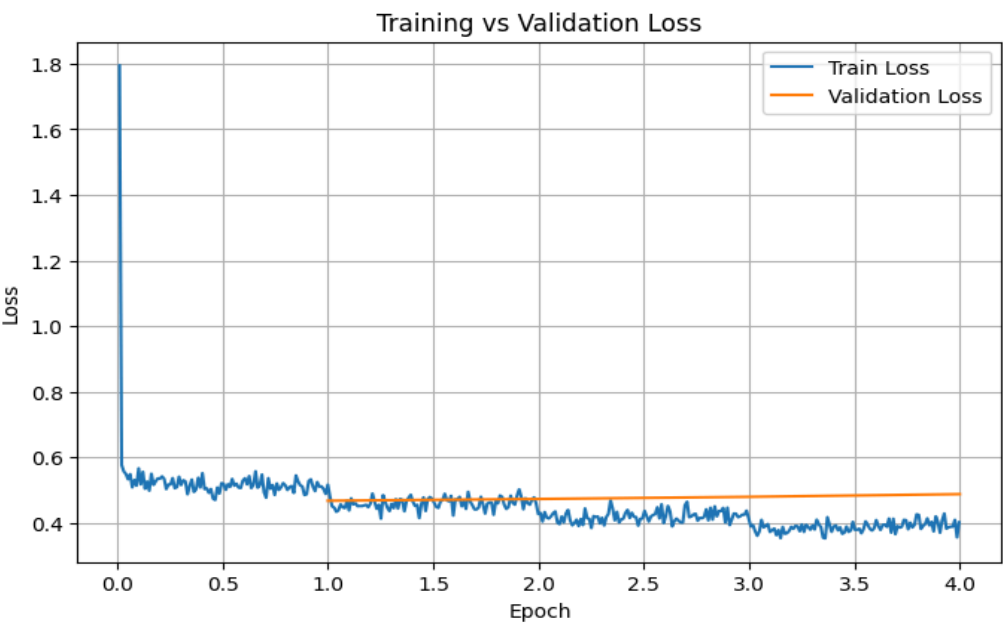
Oleh karena itu, mekanisme Early Stopping menghentikan proses pelatihan pada epoch ke-10 dan mengembalikan bobot model terbaik yang diperoleh pada epoch ke-5. Dengan demikian, model yang digunakan pada tahap evaluasi merupakan model dengan validation loss terendah sebesar 0,9995.

Tabel 3.1 Hasil Pelatihan Transformer from Scratch

Metrik	Nilai
Best Epoch	5
Training Loss Akhir	0,5635
Validation Loss Terbaik	0,9995
Total Epoch	10

3.2.2 Hasil Pelatihan MarianMT Fine-Tuning

Gambar 3.3 Grafik Loss Hasil Pelatihan Transformer from Scratch



Berdasarkan hasil fine-tuning, model MarianMT menunjukkan proses konvergen yang relatif cepat. Nilai training loss menurun dari 0,5157 pada epoch pertama

menjadi 0,4009 pada epoch keempat. Penurunan ini menunjukkan bahwa model berhasil menyesuaikan parameter pre-trained terhadap dataset yang digunakan.

Akan tetapi, validation loss mencapai nilai terendah sebesar 0,4667 pada saat epoch pertama. Setelah itu, nilai validation loss mengalami peningkatan bertahap menjadi 0,4869 pada epoch keempat. Hal ini menunjukkan bahwa proses adaptasi model terhadap dataset telah terjadi pada epoch awal sehingga pelatihan yang terlalu lama berpotensi menyebabkan overfitting.

Fenomena ini cukup umum pada model pre-trained karena parameter model telah memiliki representasi bahasa yang baik sejak awal. Akibatnya, proses fine-tuning hanya memerlukan penyesuaian kecil terhadap dataset target. Dibandingkan Transformer from Scratch, MarianMT mampu mencapai nilai validation loss yang jauh lebih rendah dengan jumlah epoch yang lebih sedikit.

Tabel 3.2 Hasil Pelatihan Transformer from Scratch

Metrik	Nilai
Best Epoch	1
Training Loss Akhir	0,4009
Validation Loss Terbaik	0,4667
Total Epoch	4

3.3 Evaluasi dan Analisis Hasil Penerjemahan

Evaluasi dilakukan untuk mengukur kualitas hasil terjemahan yang dihasilkan oleh kedua model. Pengukuran kuantitatif dilakukan menggunakan BLEU Score, sedangkan analisis kualitatif dilakukan dengan membandingkan hasil prediksi model terhadap kalimat referensi pada beberapa contoh data *training*.

3.3.1 Transformer from Scratch

Model Transformer from Scratch memperoleh BLEU Score sebesar 17,79. Nilai tersebut menunjukkan bahwa model telah mampu mempelajari hubungan dasar antara bahasa Inggris dan bahasa Indonesia, namun kualitas terjemahan yang dihasilkan masih tergolong terbatas.

Table 3.3 Tabel perbandingan Input, Referensi, Prediksi Transformer from Scratch

Input	Reference	Prediction
that thing, i'm not sure, but i think it was an acheri.	mahkluk itu, aku tidak yakin, tapi aku pikir itu adalah acheri.	itu, aku tidak yakin, tapi kurasa itu aku pikir itu
please say yes.	kumohon katakan ya.	kumohon bilang ya.
yeah... sure...	yeah... tentu saja...	ya...
i think we did something to it.	kurasa kita harus melakukan sesuatu..	kurasa kita melakukan sesuatu untuk itu.
give me the wallet!	selamatkan anak anak!	berikan aku dompet!

Berdasarkan hasil pengamatan, terdapat beberapa jenis kesalahan yang masih sering muncul. Pada contoh pertama terlihat adanya pengulangan frasa "*itu aku pikir itu*" yang menunjukkan bahwa decoder belum mampu menjaga urutan kata secara optimal. Selain itu, informasi penting berupa kata "*acheri*" juga hilang dari hasil terjemahan.

Pada contoh ketiga, model hanya menghasilkan terjemahan "*ya...*" dan tidak menerjemahkan keseluruhan kalimat. Hal ini mengindikasikan bahwa model cenderung memprediksi token akhir kalimat (EOS) terlalu cepat sehingga proses decoding berhenti sebelum seluruh informasi diterjemahkan.

Meskipun demikian, terdapat pula beberapa hasil yang cukup baik, seperti pada kalimat *"please say yes"* yang berhasil diterjemahkan menjadi *"kumohon bilang ya"*. Walaupun tidak identik dengan referensi, makna yang disampaikan masih dapat dipahami dengan baik.

Secara keseluruhan, model Transformer from Scratch telah mampu mempelajari pola dasar penerjemahan, namun masih mengalami kesulitan dalam menjaga koherensi kalimat, menghindari pengulangan kata, dan menghasilkan terjemahan yang lengkap.

3.3.2 MarianMT Fine-Tuning

Model MarianMT yang di-fine-tuning memperoleh BLEU Score sebesar 35,31 yang menunjukkan peningkatan performa yang cukup signifikan dibandingkan Transformer from Scratch.

Tabel 3.4 Tabel perbandingan Input, Referensi, Prediksi Transformer MarianMT Fine-Tuning

Input	Reference	Prediction
that thing, i'm not sure, but i think it was an acheri.	mahkluk itu, aku tidak yakin, tapi aku pikir itu adalah acheri.	hal itu, saya tidak yakin, tapi saya pikir itu adalah acheri.
please say yes.	kumohon katakan ya.	tolong katakan ya.
yeah... sure...	yeah... tentu saja...	ya... tentu...
i think we did something to it.	kurasa kita harus melakukan sesuatu..	aku pikir kita melakukan sesuatu untuk itu.
give me the wallet!	selamatkan anak anak!	berikan dompet itu!

Secara umum, MarianMT menghasilkan terjemahan yang lebih lengkap dan lebih *natural* dibandingkan Transformer from Scratch. Pada contoh pertama, model

berhasil mempertahankan informasi penting seperti kata "*acheri*" dan menghasilkan struktur kalimat yang lebih *natural*.

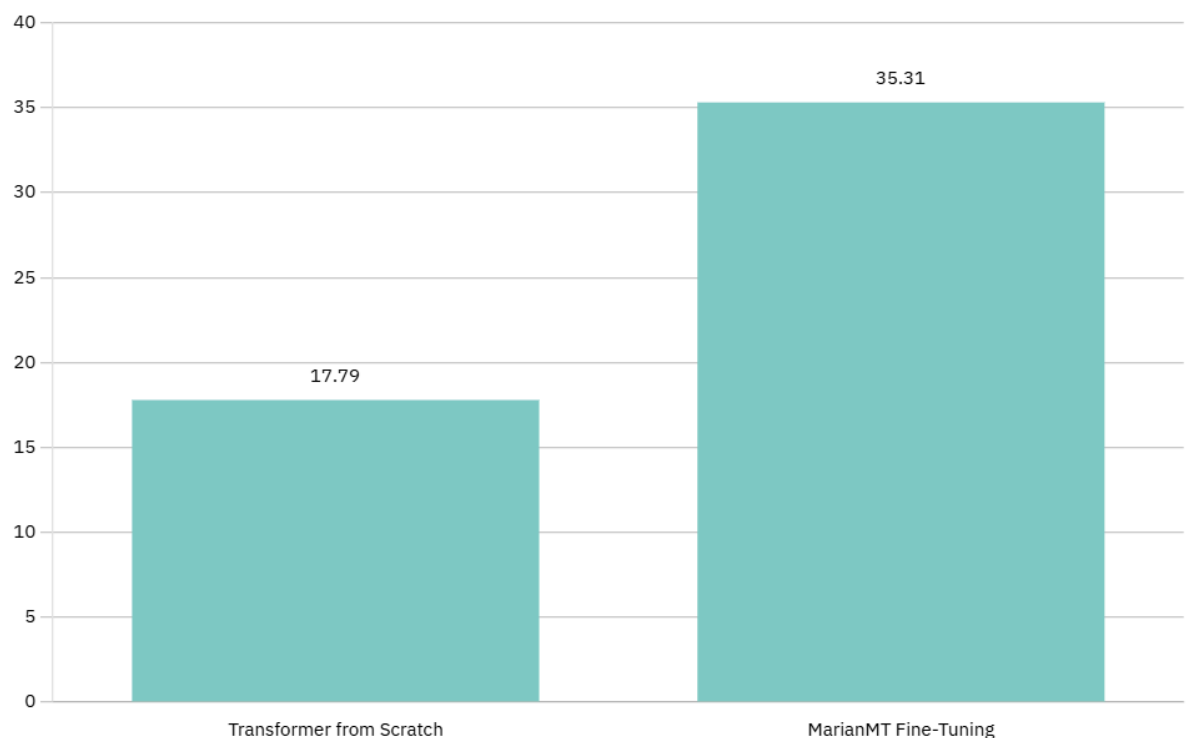
Beberapa perbedaan dengan referensi masih ditemukan, misalnya penggunaan frasa "*hal itu*" dibandingkan "*mahkluk itu*" atau "*tolong*" dibandingkan "*kumohon*". Namun perbedaan tersebut tidak mengubah makna utama kalimat sehingga masih dapat dianggap sebagai terjemahan yang benar secara makna.

Menariknya, terdapat beberapa kasus di mana hasil prediksi justru lebih sesuai dengan kalimat sumber dibandingkan referensi. Sebagai contoh, kalimat "*give me the wallet!*" diterjemahkan menjadi "*berikan dompet itu!*", sedangkan referensi yang tersedia adalah "*selamatkan anak anak!*". Hal ini menunjukkan adanya kemungkinan *noise* atau ketidaksesuaian pada beberapa data referensi.

Secara keseluruhan, MarianMT mampu menghasilkan terjemahan yang lebih natural, lebih lengkap, dan lebih stabil dibandingkan model yang dilatih dari awal.

3.3.3 Perbandingan Kedua Model

Grafik 3.1 Perbandingan Bleu Score



Berdasarkan hasil evaluasi pada grafik 3.1, MarianMT menghasilkan BLEU Score yang jauh lebih tinggi dibandingkan Transformer from Scratch. Jika dihitung, peningkatan performa mencapai sekitar 98,5% dibandingkan model yang dilatih dari awal. Hasil ini menunjukkan bahwa pemanfaatan model pre-trained memberikan keuntungan yang lebih baik dalam tugas penerjemahan bahasa.

Selain berdasarkan BLEU Score, perbedaan kualitas juga terlihat dari hasil evaluasi manual. Transformer from Scratch masih sering mengalami pengulangan kata, kehilangan sebagian informasi penting, dan menghasilkan kalimat yang kurang natural. Sebaliknya, MarianMT mampu mempertahankan konteks kalimat dengan lebih baik serta menghasilkan struktur kalimat yang lebih terstruktur.

Meskipun demikian, BLEU Score tidak selalu mampu menggambarkan kualitas terjemahan secara menyeluruh. Perbedaan pilihan kata yang sebenarnya merupakan sinonim tetap dapat menyebabkan penurunan skor. Selain itu, keberadaan data referensi yang mengandung noise/ketidaksesuaian referensi juga dapat memengaruhi hasil evaluasi. Oleh karena itu, penggunaan BLEU Score sebaiknya dilengkapi dengan evaluasi manual untuk memperoleh gambaran kualitas penerjemahan yang lebih baik.

BAB 4. Kesimpulan

4.1 Kesimpulan

Berdasarkan hasil proyek yang telah dilakukan, dapat diambil beberapa kesimpulan sebagai berikut:

- Model MarianMT yang di-fine-tuning memberikan performa yang lebih baik dibandingkan Transformer yang dibangun dari nol untuk tugas penerjemahan bahasa Inggris ke bahasa Indonesia. Hal ini ditunjukkan oleh nilai BLEU Score yang lebih tinggi, yaitu 35,31, sedangkan Transformer Scratch memperoleh BLEU Score 17,79. Selain itu, hasil evaluasi secara manual menunjukkan bahwa MarianMT menghasilkan terjemahan yang lebih natural, lebih lengkap, dan lebih konsisten dalam mempertahankan makna kalimat sumber.
- Hasil tuning hyperparameter menunjukkan bahwa konfigurasi terbaik untuk Transformer Scratch adalah 2 layer dengan learning rate 0.0005, sedangkan konfigurasi terbaik untuk MarianMT adalah learning rate 0.00001 dan weight decay 0. Pada Transformer Scratch ditemukan bahwa penambahan jumlah layer tidak selalu meningkatkan performa model. Model dengan jumlah layer yang lebih sedikit justru menghasilkan validation loss yang lebih rendah, yang kemungkinan dipengaruhi oleh kualitas dataset yang masih mengandung noise dan pasangan kalimat yang kurang selaras.
- Evaluasi menggunakan BLEU Score memberikan gambaran kuantitatif mengenai kualitas terjemahan, tapi metrik ini belum sepenuhnya mampu merepresentasikan kualitas terjemahan yang sebenarnya. Beberapa hasil terjemahan yang benar secara makna tetap memperoleh nilai lebih rendah karena adanya perbedaan pilihan kata dengan kalimat referensi. Oleh karena itu, evaluasi translasi sebaiknya juga dilengkapi dengan penilaian secara manual (human evaluation).

4.2 Saran

Berdasarkan hasil proyek yang diperoleh, beberapa saran untuk pengembangan proyek selanjutnya adalah sebagai berikut:

- Melakukan pembersihan dataset yang lebih menyeluruh untuk mengurangi noise dan pasangan kalimat yang tidak selaras.
- Menambahkan data paralel bahasa Inggris-Indonesia yang memiliki kualitas lebih baik agar model dapat mempelajari pola translasi yang lebih akurat.
- Melakukan eksplorasi hyperparameter yang lebih luas, seperti batch size, dropout, beam width, dan jumlah epoch.
- Menggunakan teknik decoding yang lebih optimal, misalnya beam search dengan jumlah beam yang lebih besar
- Mencoba model translasi yang lebih modern, seperti mBART atau NLLB, untuk membandingkan performanya dengan MarianMT.

Meskipun pengembangan tersebut berpotensi meningkatkan performa model, kualitas dataset tetap menjadi faktor yang sangat menentukan.

DAFTAR PUSTAKA

- Junczys-Dowmunt, M., Grundkiewicz, R., Dwojak, T., Hoang, H., Heafield, K., Neckermann, T., Seide, F., Hermann, U., Fikri Aji, A., Bogoychev, N., Martins, A. F. T., & Birch, A. (2018). Marian: Fast neural machine translation in C++. *Proceedings of ACL 2018, System Demonstrations*, 116–121. <https://doi.org/10.18653/v1/P18-4020>
- Kudo, T., & Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 66–71. <https://doi.org/10.18653/v1/D18-2012>
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*. <https://arxiv.org/abs/1301.3781>
- Tiedemann, J., & Thottingal, S. (2020). OPUS-MT—Building open translation services for the world. *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation*, 479–480.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30. <https://papers.nips.cc/paper/7181-attention-is-all-you-need>
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., ... Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38–45. <https://doi.org/10.18653/v1/2020.emnlp-demos.6>